

MiniOrangeOS 操作系统设计与实现

操作系统课程设计项目文档

课程名称：操作系统课程设计

项目名称：MiniOrangeOS 操作系统设计与实现

姓名：李晨恺

学号：2453875

组号：98

指导教师：王冬青

完成日期：2026 年 8 月 26 日

源码托管：https://github.com/Phantom-Lucas/orange_os.git

摘要

MiniOrangeOS 是一个面向 x86-64 与 QEMU 的单核操作系统。系统从 512 字节 MBR 开始，经 MBR/二级 Loader 协作完成内核读取、E820 内存探测、临时页表建立和长模式切换，随后进入高半区内核。内核实现 GDT/TSS/IDT、中断与异常、物理和虚拟内存、Ring3 用户态、抢占式线程调度、进程生命周期、ELF 加载、fork/exec/wait、写时复制、匿名 mmap、消息 IPC、futex 及用户态同步原语、ATA PIO LBA48、MyFS、TTY、三控制台和系统调用。用户态提供启动代码、轻量 libc、交互式 Shell、管道、重定向、后台执行和二十余个程序。

本项目参考《Orange'S：一个操作系统的实现》的路线，但没有把书中 32 位源码直接作为最终结构，而是围绕 64 位执行模式、每进程独立页表、现代系统调用 ABI、文件描述符组合和自动化测试重新组织。为解决内核增长导致 Loader 静默截断的问题，构建系统会根据 kernel.bin 的实际大小计算扇区数，Loader 支持超过 255 扇区的 ATA 分块读取，并同时检查内核磁盘区域和含 .bss 的物理内存上界。键盘中断会排空 8042 输出缓冲，降低快速输入时的漏字风险。

本操作系统通过了功能测试和性能测试。

目录

第 1 章	项目概述	7
1.1	项目目标	7
1.2	已实现功能概览	7
1.3	项目目录结构	7
第 2 章	开发环境与项目构建	8
2.1	宿主环境	8
2.2	编译与运行工具	8
2.3	依赖安装与源码获取	8
2.4	构建流程、磁盘映像生成与系统运行	9
2.5	QEMU 启动方法	9
2.6	关键构建配置	9
第 3 章	需求分析与总体设计	10
3.1	功能需求	10
3.2	系统总体架构	10
3.3	Ring0 与 Ring3 分层	10
3.4	系统初始化流程	10
3.5	磁盘布局	10
3.6	进程与线程资源模型	11
3.7	关键数据流	11
3.8	其他关键设计	11
第 4 章	MBR、Loader 与长模式启动	12
4.1	BIOS 启动过程	12
4.2	MBR 设计	12
4.3	二级 Loader 设计	12
4.4	内核磁盘布局	12

4.5 内核动态扇区计算	12
4.6 ATA 分块读取	12
4.7 A20 与长模式切换.....	13
4.8 临时页表建立.....	13
4.9 高半区跳转	13
4.10 bss 初始化.....	13
4.11 启动边界检查	13
第 5 章 内核基础设施.....	13
5.1 内核入口与初始化顺序.....	13
5.2 链接脚本和内存布局.....	13
5.3 GDT 与 TSS.....	14
5.4 IDT 设计	14
5.5 时钟中断	14
5.6 键盘中断	14
5.7 异常处理.....	14
5.8 Ring3 故障隔离	14
5.9 内核日志和 panic	14
5.10 中断现场与特权级判断.....	14
5.11 PIC、EOI 与中断处理约束	15
第 6 章 内存管理.....	15
6.1 物理页帧管理.....	15
6.2 四级页表	15
6.3 内核与用户地址空间.....	16
6.4 页面权限	16
6.5 物理页引用计数	16
6.6 用户内存访问检查	16

6.7 VMA 管理	16
6.8 匿名 mmap/munmap.....	16
6.9 用户栈按页增长	17
6.10 页错误处理	17
6.11 PMM 位图、所有者与引用三层状态	17
6.12 map_page、用户复制与回滚规则	17
6.13 COW 故障算法与失败路径	18
第 7 章 进程、线程与调度	18
7.1 进程控制块	18
7.2 线程控制块	18
7.3 抢占式调度	18
7.4 上下文切换	18
7.5 fork 实现、资源复制与回滚, COW 实现.....	18
7.6 exec、exit、wait 实现	19
7.7 进程观察与终止	19
7.8 用户线程	19
7.9 调度环与状态不变量	19
7.10 阻塞、唤醒与线程生命周期	20
7.11 ELF 进程和用户线程栈布局	20
第 8 章 系统调用、IPC 与同步	20
8.1 系统调用总体设计	20
8.2 系统调用入口、栈切换与参数安全	20
8.3 36 个系统调用分类	21
8.4 消息 IPC 与阻塞唤醒	21
8.5 mutex 与用户态同步	21
8.6 系统调用入口栈帧	22

8.7 IPC、futex 与锁边界.....	22
第 9 章 ATA 驱动与 MyFS 文件系统.....	23
9.1 ATA PIO 与 LBA48 驱动.....	23
9.2 MyFS 磁盘格式.....	23
9.3 直接和多级间接索引.....	24
9.4 路径、文件与目录操作.....	24
9.5 文件对象、描述符与管道.....	24
9.6 FS 服务与宿主机 mkfs.....	24
9.7 关键算法、缓存与服务协议.....	25
第 10 章 TTY、控制台与显示系统.....	27
10.1 TTY 总体结构.....	27
10.2 键盘输入.....	27
10.3 虚拟控制台与滚屏.....	27
10.4 VGA、framebuffer 与字体渲染.....	27
10.5 终端编辑与前台进程控制.....	28
10.6 键盘事件与 TTY 输入服务.....	28
10.7 ANSI、历史与重绘模型.....	28
10.8 framebuffer 渲染与前台控制键.....	28
第 11 章 用户态、libc 与 Shell.....	29
11.1 用户程序启动与系统调用封装.....	29
11.2 libc.....	29
11.3 Shell、内建命令与外部程序.....	29
11.4 管道、重定向与后台任务.....	30
11.5 Shell 行编辑、历史与补全.....	30
11.6 用户态工具.....	30
11.7 Shell 解析与外部命令执行.....	30

第 12 章	系统测试与结果分析.....	31
12.1	测试目标与框架.....	31
12.2	fast/full/stress 分层.....	31
12.3	构建、启动与存储测试.....	32
12.4	用户态、输入与虚拟内存测试.....	32
12.5	同步、显示与 guided tour 测试.....	33

第1章 项目概述

1.1 项目目标

项目目标分为五个闭环：

1. 裸机自举：从自己的 MBR 和 Loader 进入 64 位内核。
2. 用户隔离：建立 Ring3、独立地址空间、系统调用和用户异常隔离。
3. 系统并发：实现进程、线程、调度、IPC、futex 和用户态同步。
4. 持久交互：实现磁盘、文件系统、TTY、显示、Shell 和用户应用。
5. 可重复验证：用构建检查、黑盒交互、边界和压力测试形成证据链。

1.2 已实现功能概览

领域	已实现功能
启动	MBR、二级 Loader、E820 内存探测、A20、页表、长模式、高半区、动态内核扇区
内核基础	GDT、TSS、IDT、时钟、键盘、异常、panic、启动日志
内存	页帧、四级页表、独立 CR3、引用计数、COW、VMA、匿名 mmap
进程线程	spawn、fork、exec、exit、wait、kill、ps、用户线程、TLS、抢占调度
ABI/并发	36 个系统调用、消息 IPC、futex、mutex、condvar、pipe
存储	ATA PIO LBA48、MyFS、层级目录、三级间接索引、宿主 mkfs
终端	TTY、三控制台、滚屏、编辑、framebuffer、VGA 回退
用户态	crt0、libc、Shell、argv、管道、重定向、后台、20 余个程序
验证	fast/full/stress、16 项完整回归、100 轮同步压力、截图与 artifact

1.3 项目目录结构

my_os/

├── boot/MBR 和二级 Loader

├── kernel/内核、驱动、内存、进程、FS、TTY、显示

└── usr/用户启动代码、libc、Shell、工具和演示程序

- └──tools/宿主机 MyFS 格式化工具
- └──tests/统一测试 runner、manifest、suite 和 QEMU 工具
- └──assets/framebuffer 使用的字体等资源
- └──design/最终项目文档与答辩材料
- └──Makefile 构建、镜像、运行、测试和展示入口
- └──build/生成制品和测试 artifact，不作为源码提交

第2章 开发环境与项目构建

2.1 宿主环境

项目	已验证值
操作系统	Ubuntu 22.04.5 LTS x86-64
Linux 内核	6.8.0-136-generic
编译器	GCCx86-64
汇编器	NASM
虚拟机	qemu-system-x86_64
Guest 配置	1CPU，默认 1GiB；展示 512MiB

2.2 编译与运行工具

1. GCC：编译内核和用户程序。
2. NASM：生成 MBR、Loader 和部分上下文切换代码。
3. QEMU：模拟 x86-64CPU、ATA、VGA/Bochs 显示设备和内存。
4. GNU Make：统一管理配置、依赖、镜像布局、测试和演示。
5. GDB：通过 QEMU gdb stub 调试内核 ELF 符号。

2.3 依赖安装与源码获取

Ubuntu 22.04 可执行：

```
sudo apt update
```



```
sudo apt install build-essential binutils nasm make qemu-system-x86coreutilsgit
```

```
git clone https://github.com/Phantom-Lucas/orange_os.git
```

2.4 构建流程、磁盘映像生成与系统运行

构建流程

```
make build
```

```
make check
```

构建完成后主要制品位于：

build/mbr.bin、build/loader.bin、build/kernel/kernel.elf、build/kernel/kernel.bin、build/*.elf、
build/mkfs、build/fs.img、build/images/orange-dev.img

make check：检查 MBR 大小与签名、Loader 大小、内核磁盘扇区容量、含 BSS 的物理末端、
用户 ELF 和 MyFS 制品。检查失败必须修复布局或实现，不能简单删除检查。

磁盘镜像生成首次运行：

make bootstrap：该目标创建磁盘、写 MBR/Loader/内核并格式化 MyFS。format-fs 会覆盖目
标镜像的 MyFS 区域，已有 Guest 文件会丢失。只修改内核且希望保留文件时使用 makerun，
它只安装启动和内核区。

2.5 QEMU 启动方法

```
make run
```

支持的分辨率：make FB_MODE=1024x768 run、make FB_MODE=1280x720 run、make
FB_MODE=1440x900 run

2.6 关键构建配置

变量	默认值	作用
DISK_SIZE	256M	开发磁盘大小
FS_START_LBA	1000	MyFS 起始扇区
KERNEL_START_LBA	10	内核起始扇区
QEMU_MEMORY	1G	Guest 内存
QEMU_CPUS	1	单核设计约束

CONSOLE_BACKEND	qemu-fb	默认交互显示后端
FB_MODE	1280x720	framebuffer 模式
BOOT_DIAGNOSTIC	0	启动详细日志开关
SYNC_TEST_ROUNDS	10	full 同步轮数
SYNC_WORKER_ROUNDS	20000	每线程共享更新次数

Makefile 用配置 stamp 把编译宏写入依赖图。配置变化时必须重编相关对象，避免增量构建错误复用另一种 framebuffer/diagnostic/FS 起点的内核。

第3章 需求分析与总体设计

3.1 功能需求

系统启动需求包括 BIOS 自举、磁盘读取、长模式和内核入口；内核需求包括异常、中断、内存、调度、进程、线程、系统调用、IPC、存储与终端；用户态需求包括 ELF、libc、Shell、文件工具和演示程序。验证需求包括构建边界、功能、持久化、压力、输入和显示测试。

3.2 系统总体架构

BIOS/QEMU、MBR 512B、二级 Loader、x86-64 高半区内核、异常/时钟/键盘、页帧/页表/VMA/COW、进程/线程/调度、消息/futex/同步、ATA PIOLBA48、MyFS/FS 服务、TTY/控制台/framebuffer、36 个系统调用、用户态 libc、Shell、用户程序/guidedtour

3.3 Ring0 与 Ring3 分层

内核、驱动、页表管理和服务线程运行在 Ring0。Shell 与普通应用运行在 Ring3，只能通过系统调用访问内核资源。GDT/TSS、页表 U/S 权限与系统调用入口共同建立保护边界。用户异常被转换为进程退出，内核异常才进入 panic。

3.4 系统初始化流程

BSS → print → IDT → PIC → timer → GDT → PMM → kmalloc → TTY → keyboard → thread → futex → IPC → TTY service → syscall → sti → disk → MyFS → FS service → framebuffer → 用户程序。

初始化必须满足依赖顺序。

3.5 磁盘布局

区域	默认 LBA	用途
MBR	0	BIOS 首阶段
Loader	2 起	二级启动代码
Kernel	10 起	按实际大小写入和加载
MyFS	1000 起	超级块、位图、inode 表和数据

内核最大磁盘扇区数为 FS_START_LBA-KERNEL_START_LBA，默认 990。物理加载区的另一上界是 0x70000，该处开始放置启动页表。两者分别约束磁盘和运行时物理内存。

3.6 进程与线程资源模型

进程保存 PID、状态、父子关系、CR3、VMA、当前目录、文件描述符表和线程链表。线程保存 TID、调度上下文、内核栈、用户栈、TLS、优先级和阻塞状态。一个进程的线程共享地址空间和文件表，但拥有独立执行现场。

3.7 关键数据流

文件读取：Ring3read→copyin 参数→fileobject→FSrequest→ATA→copyout

键盘输入：8042IRQ→scancode decode→TTYqueue→foregroundShellread

Shell 管道：pipe→fork→dup2→exec→fileobjectbuffer→wait/background

COW 写入：writefault→PTE/COW 检查→refcount→copy/remap→TLBrefresh

3.8 其他关键设计

1. 磁盘布局：MBR/Loader/Kernel/MyFS 区域不能重叠。
2. 物理布局：内核含 BSS 不能覆盖启动页表和 PMM 位图。
3. 权限：用户 PTE 不得映射内核私有页，系统调用不得直接信任用户指针。
4. 调度：调度环最多一个 RUNNING；BLOCKED 不可被当 READY 切入。
5. COW：所有共享 PTE 引用与 PMMrefcount 相等，写页必须只读+COW。
6. 生命周期：zombie 只回收一次；文件、pipe、futex、IPC 等待都在退出时解除。
7. 锁顺序：address-space、heap、PMM 及 bucket/file 锁按约定顺序获取。
8. 文件格式：mkfs 与内核对超级块、inode、目录项和块号含义一致。
9. IRQ：中断路径有界且不睡眠；EOI 在设备数据转移后发送。

第4章 MBR、Loader 与长模式启动

4.1 BIOS 启动过程

QEMU 以 BIOS 模式启动后把磁盘 LBA0 的 512 字节加载到 0x7C00 并进入实模式代码。MBR 必须在最后两个字节包含 0x55AA，并在极小空间内完成段寄存器/栈初始化和第一批 ATA 读取。

4.2 MBR 设计

boot/mbr.S 是第一阶段入口。它直接使用 ATA PIO LBA28 兼容命令，从 LBA2 连续读取 50 个扇区到物理 0x900：其中 LBA2 ~ 9 是为 Loader 预留的 8 扇区窗口，LBA10 ~ 51 是内核前 42 扇区。随后跳到 0x900。MBR 不读取完整内核，剩余尾部由已经脱离 0x7C00 的 Loader 继续加载。这样既利用首批顺序读取，也避免写目标覆盖正在执行的 MBR。

4.3 二级 Loader 设计

boot/loader.S 负责继续读取内核、用 BIOSE820 探测物理内存、打开 A20、准备 GDT 和启动页表、设置控制寄存器与 EFER，最终进入 64 位入口并跳向内核。Loader 本身仍受构建大小检查保护。

4.4 内核磁盘布局

内核从 LBA10 开始，MyFS 默认从 LBA1000 开始。构建先计算 kernel.bin 的字节数和向上取整扇区数；若超过 990 个可用扇区，构建直接失败，而不是覆盖文件系统。

4.5 内核动态扇区计算

核心计算为：

```
kernel_sectors=(kernel_bytes+511)/512
```

```
kernel_max_sectors=FS_START_LBA-KERNEL_START_LBA
```

Makefile 通过 -DKERNEL_SECTORS=... 把结果传给 NASM。Loader 只读取实际需要的尾部扇区，不再使用固定 188 扇区窗口。默认预加载数为 42 扇区，实际内核超过后再读取剩余部分。

4.6 ATA 分块读取

一次 ATA 扇区计数命令的有效范围有限。Loader 的读取包装器把大请求拆成不超过 255 扇区的块，更新 LBA、目标内存地址和剩余计数，直到读取完成。因此 300 扇区测试会走过一个 255 扇区块和尾部块，而不是发生计数回绕。

4.7 A20 与长模式切换

打开 A20 后, Loader 准备长模式所需的 PAE 页表, 设置 CR4.PAE、EFER.LME 和 CR0.PG, 加载 64 位 GDT 并通过远跳转进入长模式。切换顺序不能颠倒, 否则 CPU 会产生异常或三重故障。

4.8 临时页表建立

启动页表位于约 0x70000 的预留物理区域, 提供内核进入时所需的恒等/高半区映射。该区域不能被内核镜像或 BSS 覆盖, 因此仅检查 kernel.bin 文件长度是不够的。

4.9 高半区跳转

kernel/linker.ld 定义内核链接虚拟地址与对应物理加载地址。Loader 在页表生效后跳到 64 位内核入口, 内核代码按高半区符号运行。所有用户进程页表共享必要的内核高半区映射。

4.10 bss 初始化

链接脚本导出 __bss_start、__bss_end 和 _kernel_phys_end。内核最早入口在使用锁、队列和全局指针前把 BSS 置零。这样不依赖 objcopy 原始镜像包含未初始化段的全部零字节。

4.11 启动边界检查

构建有两类独立检查:

1. 磁盘边界: 内核扇区不得到达 MyFS 起始 LBA。
2. 物理边界: __kernel_phys_end 不得越过 0x70000 启动页表。

磁盘文件大小和运行时内存占用不是同一个概念, 两项检查不能合并为一个固定常数。

第5章 内核基础设施

5.1 内核入口与初始化顺序

kernel/kernel.c 的入口首先清零 BSS, 再初始化输出、内存、描述符表、中断、调度、系统调用、设备、FS 和用户态。初始化日志分为必要摘要与可选诊断, BOOT_DIAGNOSTIC=1 用于展开启动细节, 默认模式保持终端简洁。

5.2 链接脚本和内存布局

kernel/linker.ld 控制代码、只读数据、数据和 BSS 的顺序, 导出启动和边界所需符号。链接 ELF 保留调试符号, Loader 使用 objcopy 产生的原始 BIN; 两者用途不同, GDB 应加载

ELF。

5.3 GDT 与 TSS

GDT 定义 64 位内核代码/数据和用户代码/数据描述符。TSS 提供从 Ring3 进入 Ring0 时的栈信息。用户态进入和系统调用/中断返回都依赖选择子与权限级正确配置。

5.4 IDT 设计

IDT 为 CPU 异常、时钟和键盘建立门描述符。汇编入口保存现场后调用 C 处理函数，再按中断来源恢复现场。用户态异常与内核异常需要依据 CS/错误码和当前执行上下文区分。

5.5 时钟中断

时钟中断更新系统 tick，并为抢占式调度提供时机。阻塞操作不应持续忙等占用 CPU，而应改变线程状态并让调度器选择其他可运行线程。

5.6 键盘中断

键盘 IRQ 读取 8042 状态和数据端口，将扫描码送入解码/队列。当前 ISR 会在单次中断中排空所有已就绪字节，并设置 64 字节上限，之后再向 PIC 发送 EOI。该设计解决渲染或系统调用繁忙期间扫描码积压造成的漏字和乱序。

5.7 异常处理

页错误处理先判断是否属于 COW，或是否命中允许向下增长的用户栈 VMA；可以修复则更新页表继续执行。非法用户访问转为进程退出，内核态不可恢复异常输出寄存器和错误信息后 panic。

5.8 Ring3 故障隔离

usr/fault.c 故意触发非法访问。Shell 通过 spawn/wait 观察子进程退出，随后仍显示 prompt。showcase.elf 也在第六步执行该程序，要求状态不小于约定异常退出基值且演示继续。

5.9 内核日志和 panic

正常启动日志应简洁，避免测试依赖大量调试输出。panic 用于内核不变量破坏，不作为普通用户错误处理方式。Shell 的 panic/fault 类命令只用于开发验证，不应在正式演示中误操作内核 panic。

5.10 中断现场与特权级判断

异常/IRQ 入口必须保存足够现场，并根据 frame->cs&3 判断是否来自用户态。来自 Ring3

时，硬件中断帧还包含用户 RSP/SS；内核记录线程的用户 RSP，并处理 GS 基址切换。若中断期间调度切换到别的线程，旧入口不能再用旧假设恢复新线程现场。

页错误的关键输入为 CR2 和 errorcode：

位	含义	本项目用途
0	0=页不存在，1=保护违规	区分栈增长与 COW/非法权限
1	1=写访问	COW 必须由写触发；检查 VMA_WRITE
2	1=用户态	决定终止进程还是内核 panic

5.11 PIC、EOI 与中断处理约束

PIC 初始化后把硬件 IRQ 映射到 IDT32 起，时钟为 32、键盘为 33。中断处理完成后必须发送 EOI；但键盘需要先读完控制器缓冲和转移事件，再确认 PIC。IRQ 中不能调用可能睡眠的 FS、用户复制或 mutex 路径，输入仅入队，后续由 TTY 服务消费。

spinlock 获取期间会屏蔽本地中断，避免单核上同一 CPU 的中断处理器重入并等待自己持有的锁。这不等同于 SMP 锁正确性，项目仍限定 1CPU。

第6章 内存管理

6.1 物理页帧管理

物理内存管理器以 4KB 页为基本单位，记录可用页帧、占用状态和共享引用。内核代码、启动页表、设备区域和其他保留物理范围不能进入普通分配池。页分配失败必须沿调用链返回或触发明确的内核错误，不能返回可能覆盖已有对象的地址。

物理页接口供页表、进程地址空间、用户栈、内核栈、匿名映射和 COW 使用。释放前要确认页的所有权和引用计数，避免同一页重复释放。

6.2 四级页表

x86-64 使用 PML4、PDPT、PD 和 PT 四级结构。虚拟地址由各级索引和页内偏移组成。页表层提供创建映射、查询 PTE、修改权限、取消映射和销毁用户地址空间等操作。新建中间页表时必须清零，避免随机位被解释为 Present 或权限标志。

Virtual Address

PML4 9b	PDPT 9b	PD 9b	PT 9b	Offset 12b
---------	---------	-------	-------	------------

6.3 内核与用户地址空间

每个用户进程具有独立 CR3 和用户区映射，内核高半区映射由所有进程共享。用户页设置 U/S 权限，内核页不允许 Ring3 访问。切换进程时调度器加载目标 CR3；同进程线程共享 CR3。

这一模型使相同虚拟地址可以在不同进程映射到不同物理页，也使非法用户指针无法仅凭数值进入内核区域。

6.4 页面权限

PTE 的 Present、Writable、User 等位决定硬件访问行为。ELF 代码、只读数据、可写数据和栈应按段权限映射。COW 借用只读页面触发写错误，同时使用软件标志区分“合法 COW”与真正的只读违规。

6.5 物理页引用计数

普通独占页引用数为 1。fork 共享 COW 页时，父子 PTE 指向同一物理页并增加引用。子进程或父进程退出、取消映射或完成复制时减少引用；只有归零才能回收到分配器。

引用计数不变量为：物理页引用数=所有有效共享映射和内核所有者引用的总和

6.6 用户内存访问检查

系统调用不能把用户指针当作可信内核指针。实现按参数方向执行 copyin/copyout，并检查地址范围、页是否存在、用户权限和写权限。字符串还需要长度上限和终止符检查。

FS/TTY 服务可能阻塞和调度，因此系统调用层先把用户数据复制到内核暂存区，服务线程不长期保存用户地址。这样即使进程映射变化或退出，也不会异步解引用失效指针。

6.7 VMA 管理

进程的 vm_area 链表记录映射起止、读写执行权限、匿名/私有属性、预留文件信息和偏移。VMA 描述合法虚拟地址范围和权限，PTE 描述当前实际映射。普通匿名 mmap 当前立即分配，初始 ELF 用户栈则利用 VM_GROWSDOWN VMA 支持缺页时逐页增长。

6.8 匿名 mmap/munmap

当前 mmap 主要支持匿名私有映射，长度按页对齐，选择不与现有 VMA 冲突的用户地址范围，并在调用期间为整个区间分配清零物理页；任一分配或映射失败会回滚已经建立的页和 VMA。munmap 解除页表映射、减少物理页引用，并按删除前部、尾部、中段或全部四种情况修改 VMA。

6.9 用户栈按页增长

ELF 初始栈只预映射顶部一页，同时登记最多 8MiB 的 VM_GROWSDOWN VMA。用户访问尚未映射但位于该 VMA 内的栈地址时产生 non-present page fault，处理器按 VMA 权限分配物理页并建立 PTE，然后返回用户态重试指令。普通匿名 mmap 当前采用立即分配；非法 VMA 外访问结束进程。

6.10 页错误处理

页错误依据错误码区分 present/non-present、read/write 和 user/kernel。处理顺序为：

1. 若为用户写 COW 页，进入 COW 分离。
2. 若为合法 VM_GROWSDOWN 栈 VMA 的未映射页，分配并映射新栈页。
3. 若为其他用户错误，记录退出状态并终止进程。
4. 若为内核态不可恢复错误，输出诊断并 panic。

6.11 PMM 位图、所有者与引用三层状态

Bitmap 同时维护 allocated_bits、reserved_bits、owners 和 refcounts。三者含义不同：

- allocated 表示 PMM 已把页交给某个拥有者。
- reserved 表示页永远不能作为普通空闲页分配。
- owner 表示当前对象类别，如 PAGE_TABLE、HEAP、USER、THREAD、PROCESS、TTY。
- refcount 只统计用户 PTE 映射引用，不代替 owner。

释放接口带 expected owner，能发现用错误生命周期回收页。PMM 锁只保护位图、owner、引用和页清零；锁内不能 kmalloc、打印或递归创建页表。全局锁顺序约定为 address-space→heap→PMM，防止不同路径反向获取。

6.12 map_page、用户复制与回滚规则

map_page 按四级索引逐层查找，不存在的中间表由 PAGE_OWNER_PAGE_TABLE 页创建并清零；它不会覆盖已有映射。确需替换时调用显式 remap_page，避免无意泄漏旧页。用户映射成功后增加相应物理页的用户引用。

copy_from_user/copy_to_user 先验证完整范围，处理跨页时逐段复制。范围计算检查加法溢出和 48 位用户上界。read 系统调用的目标必须 writable，write 的来源只需 readable；路径使用 copy_string_from_user 并设置最大长度。

匿名 mmap 的失败回滚顺序为：解除已映射页→释放物理页/引用→从 VMA 链移除本次区域→释放 VMA 节点→返回错误。munmap 中段时提前分配 suffixVMA，保证没有足够内核内存时不先破坏原映射。

6.13 COW 故障算法与失败路径

如果新页分配失败，旧 PTE 保持只读 COW，处理函数返回错误，用户进程按 fault 路径终止；不能先减少旧引用再尝试分配。fork 克隆地址空间也采用“两阶段”策略：先完整建立子页表，成功后才把父可写页统一改为 COW，避免子表创建失败却已破坏父映射。

第7章 进程、线程与调度

7.1 进程控制块

kernel/process.h 中的 struct process 保存 PID、状态、退出码、CR3、父子/兄弟链、线程链、cwd、VMA、文件表、进程名和多把资源锁。进程状态包括运行、僵尸和死亡；子进程退出后保留最小信息，直到父进程 wait 或孤儿回收路径处理。

进程级锁按明确顺序获取，例如地址空间锁、文件表锁和 fileobject 锁不能反向嵌套，以降低死锁风险。

7.2 线程控制块

线程结构保存 TID、所属进程、调度上下文、内核栈、用户栈、TLS、优先级、状态、退出码和等待关系。系统调用或中断中发生调度时，用户返回栈必须随线程保存，不能只放在全局 CPU 临时槽中。

7.3 抢占式调度

调度器维护可运行实体，时钟中断提供抢占点。线程因 wait、IPC、futex 或服务请求阻塞时，状态从 runnable/running 转为 blocked，事件完成后重新加入可运行集合。项目为单核设计，不宣称支持多个 CPU 同时调度。

7.4 上下文切换

上下文切换保存旧线程需要恢复的寄存器和栈，选择新线程，必要时切换 CR3，然后恢复新线程。从用户态中断或系统调用切入的线程还必须维护正确的用户 RSP、GS 状态和返回帧。

7.5 fork 实现、资源复制与回滚，COW 实现

fork 复制进程级元数据、VMA 和文件对象引用，建立子页表，并复制当前系统调用返回现场。父进程得到子 PID，子进程从同一个调用点返回 0。多线程进程 fork 的语义比单线程复

杂，当前实现对允许场景保持明确限制，不宣称完整 POSIX 多线程 fork。

fork 当前要求调用进程为单线程，避免复制其他线程栈、锁状态和执行现场。复制顺序为：

1. 在父地址空间锁下克隆低半区用户映射，子 PTE 建立成功后再封存父 COW。
2. 分配子 process，复制 cwd、VMA、mmap/TLS 游标。
3. 复制文件描述符并增加 fileobject 引用，不复制文件内容。
4. 分配子 thread/内核栈，复制 syscallframe 和 callee-saved 寄存器。
5. 子上下文入口设为 syscall_child_return，令 RAX=0；父返回子 PID。
6. 链接父子关系并把子线程加入调度环。

任一阶段失败都逆序释放子页表、VMA、文件引用、线程和 process。由于父 COW 修改放在子页表完整建立之后，页表分配失败不会让父进程意外失去写权限。

父子私有可写页在 fork 后都改为只读 COW，并增加物理页引用。首次写入触发异常：多人共享时分配新页复制并减少旧页引用；只有一个引用时直接恢复写权限。最后刷新相应 TLB。

7.6 exec、exit、wait 实现

exec 在当前进程中装入新 ELF 映像，成功后替换旧用户地址空间和用户入口，不创建新 PID。实现必须先构造可用新映像，再安全释放旧映像；若中途失败，应保留可报告的错误，而不是让进程处于无地址空间状态。

exit 记录状态、终止同进程执行实体并通知等待者。父进程 wait 指定子 PID 或按接口规则等待，取得退出码后回收僵尸对象。孤儿由内核回收路径处理。Shell 通过 wait 管理前台任务，后台任务则在后续轮询/回收中处理。

7.7 进程观察与终止

ps 系统调用返回 PID、PPID、状态、线程数、用户页数和名称快照。kill 请求目标进程结束，TTY 的 Ctrl+C 则面向前台任务，不能错误地终止内核服务或后台无关进程。

7.8 用户线程

thread_create 为同进程新线程分配用户栈、内核栈和 TLS，并把入口函数及参数布置到用户现场。thread_join 等待可连接线程并取得退出码；detach 表示无需 join，退出后可自动回收；yield 主动让出执行机会。

7.9 调度环与状态不变量

调度器以循环链连接线程，从当前线程的 next 开始寻找第一个 READY 实体，跳过 BLOCKED、ZOMBIE 和 DEAD。切换前运行 `check_scheduler_invariants()`：链表节点必须同时关联 process，遍历数设置上限防止环损坏，整个环中最多有一个 RUNNING 线程。

实际切换顺序为：保存旧用户 RSP→更新 TSS.RSP0/current_cpu.kernel_rsp→必要时写 CR3→设置用户 FSbase 为新线程 TLS→规范化内核 GS→修改状态/current_thread→switch_to。CR3 只在进程地址空间不同才切换，同进程线程不需要换页表。

7.10 阻塞、唤醒与线程生命周期

调度状态和生命周期是两个正交维度：BLOCKED 线程仍可能是 joinable-running；线程退出后进入 JOINABLE_ZOMBIE 或 DETACHED_ZOMBIE。joinable 线程由唯一 joiner 取得状态并回收，detached 线程由调度器安全点自动回收。THREAD_REAPED 防止重复释放。

`thread_block_with_lock(guard)`用于管道等等待：调用者持有队列 spinlock，函数先把当前线程标为 BLOCKED，再原子地释放 guard 和调度。这样生产者不可能在“检查条件”和“进入阻塞”之间发送一个随后丢失的唤醒。

退出路径会取消 futex 等待和 IPC 关系，唤醒 joiner；若是进程最后一个活动线程，则进入 `process_exit`，关闭文件、释放地址空间、记录进程退出码并唤醒父进程。

7.11 ELF 进程和用户线程栈布局

ELF 主线程的栈顶为 `0x00007FFFFFFF000`，初始映射一页，并登记向下最多 8MiB 的栈 VMA。新用户线程从 `next_thread_stack_top` 向下分配独立一页栈，线程栈之间保留一页保护间隙；TLS 从独立地址游标分配一页，并通过 `FS.baseMSR` 激活。

内核为新线程构造 `iretframe`：SS、RSP、RFLAGS、CS、RIP，带参数线程还在约定位置放 arg。`return_to_user_with_arg` 把参数取到 RDI 后 `iretq`，满足 C 函数第一个参数约定。

第8章 系统调用、IPC 与同步

8.1 系统调用总体设计

用户程序不能直接调用内核 C 函数。`usr/syscall.h` 提供封装，参数按 x86-64 约定放入寄存器，执行 `syscall`；`kernel/syscall_entry.S` 保存现场并调用 `kernel/syscall.c` 分发器。编号在内核和用户头文件中保持一致。

8.2 系统调用入口、栈切换与参数安全

内核初始化 STAR、LSTAR、FMASK 等 MSR。`syscall` 后，硬件把用户返回 RIP 放在 RCX、标志放在 R11，但不会自动切换到预期的进程内核栈。入口通过 `swaps` 取得当前

CPU/线程数据，保存用户 RSP，再切换内核栈。

返回前恢复用户现场、用户栈和 GS 状态，并用 `sysret` 返回。所有可被调度打断的返回数据都必须属于当前线程，而不是无归属的全局变量。

Ring 3 栈内容不可信，内核不能直接在其上处理中断和系统调用。每个线程拥有独立内核栈。系统调用阻塞后可能切到其他线程，恢复时必须回到原线程自己的内核栈和用户 RSP。

系统调用最多使用项目 ABI 规定的参数寄存器。数值参数经过范围检查；指针参数根据读写方向执行 `copyin/copyout`；路径和 `argv` 有长度/数量限制。FS 请求复制到内核缓冲后才交给服务线程。

8.3 36 个系统调用分类

类别	系统调用
基础 I/O	<code>write</code> , <code>read</code> , <code>clear</code> , <code>get_ticks</code> , <code>sleep</code>
文件目录	<code>open</code> , <code>close</code> , <code>unlink</code> , <code>list</code> , <code>mkdir</code> , <code>stat</code> , <code>chdir</code> , <code>getcwd</code>
进程	<code>spawn</code> , <code>exec</code> , <code>fork</code> , <code>exit</code> , <code>wait</code> , <code>getpid</code> , <code>kill</code> , <code>ps</code>
线程	<code>thread_create</code> , <code>thread_join</code> , <code>thread_exit</code> , <code>gettid</code> , <code>thread_detach</code> , <code>thread_yield</code>
IPC/同步	<code>send</code> , <code>receive</code> , <code>futex_wait</code> , <code>futex_wake</code>
虚拟内存	<code>mmap</code> , <code>munmap</code>
文件描述符	<code>dup</code> , <code>dup2</code> , <code>pipe</code>

8.4 消息 IPC 与阻塞唤醒

IPC 消息包含来源 PID、type 和 value。发送者指定目标 PID，接收者可指定来源或按接口规则接收。目标未准备时，发送/接收可能阻塞，事件到达后由调度器唤醒。进程退出必须清理等待关系，避免永久等待已消失对象。

`guidedtour` 中父进程发送 type 与 `0xC0DE`，子进程检查来源和值并回复，父进程再 `wait` 子进程，形成往返和生命周期的端到端验证。

阻塞操作遵循：在保护队列的临界区内检查条件、登记等待、改变状态，再调度；事件端在同一同步规则下更新条件和唤醒。若“检查条件”和“进入等待队列”之间存在空窗，就会发生 `lostwakeup`。

8.5 mutex 与用户态同步

`futex` 将无竞争路径留在用户态。用户 mutex 先用原子操作尝试获得锁；只有竞争时调用

futex_wait 睡眠，解锁时调用 futex_wake 唤醒等待者。内核 wait 再次检查用户值，防止条件已经变化却仍入睡。

user_mutex_t 封装原子状态和 futex。guidedtour 创建 4 个线程，每个线程在 mutex 保护下记录 TID 并增加共享 counter；最终 counter 必须为 4，join 退出码必须为 11、12、13、14。

条件变量与 mutex 配合：等待者在不丢失通知的前提下释放 mutex、睡眠，唤醒后重新获得 mutex 并再次检查谓词。条件变量不保存业务条件本身，调用者必须用循环检查共享状态。

8.6 系统调用入口栈帧

syscall_entry 的主要现场从栈底到恢复顺序包含用户 RCX/R11、参数寄存器、callee-saved 寄存器。入口把 RAX 作为 syscall 号传给 C 的 RDI，把用户前三个参数重新排列到 RSI/RDX/RCX，并把当前内核栈上的 frame 地址作为第五个 C 参数传入，供 fork 复制。

Ring 3: RAX=no, RDI=a1, RSI=a2, RDX=a3

syscall

swapgs → save user RSP → kernel RSP

push return/argument/callee-saved registers

syscall_handler(no, a1, a2, a3, frame_rsp)

restore registers → user RSP → swapgs → sysretq

FMASK 在硬件入口屏蔽不应继承的标志，C 调用前保持 16B 栈对齐。exec 成功不回到原调用点，而通过 resume_user_image 装入新 CR3 和 iretframe；exit 同样不走普通返回。

8.7 IPC、futex 与锁边界

IPC 等待状态嵌入 struct thread，避免阻塞路径临时分配失败。receiver 维护 sender 链表；每个 sender 保存等待目标和消息副本。发送算法：

if receiver 正在接收 IPC_ANY 或当前 sender.pid:

 复制消息到 receiver，标记成功，唤醒 receiver

else:

 sender 保存 message 和 receiver

 加入 receiver.sender_queue

 sender BLOCKED

接收算法先在 sender queue 查找匹配 PID；找到则取出、复制并唤醒 sender。未找到时

保存 `receive_from` 并阻塞。进程/线程退出调用 `ipc_abort_thread/current`：从队列摘除相关节点，把 等待结果设为中断/错误并唤醒，避免悬空指针和永久等待。

futex 哈希桶与锁顺序：

内核有 64 个 futex bucket，哈希键同时包含 `process` 地址和用户虚拟地址，因此不同进程相同 虚拟地址不会互相唤醒。`waiter` 节点直接嵌在 `thread`，记录前后指针、`bucket`、`process`、`uaddr` 和 返回结果。

`wait` 的锁顺序为 `process.address_space_lock` → `bucket.lock`：先保证地址空间不会销毁，再在桶锁 内第二次 `copy` 用户值；只有值仍等于 `expected` 才把线程设 `BLOCKED` 并入队。`waker` 若先获得桶 锁，`waiter` 随后重读新值并返回 `AGAIN`；`waiter` 若先入队，`waker` 一定看得到它。这是避免 `lost wakeup` 的核心证明。

`futex_cancel_process` 也按 `address-space` → `bucket` 顺序遍历并取消该进程所有 `waiter`。统计 接口返回桶数、等待者总数和非空桶数，便于测试退出后的泄漏。

内核锁与用户同步的边界：

`spinlock` 用于不可睡眠的短临界区，并关闭本地中断；`kernel mutex` 在竞争时把线程挂到等待路径， 适合可睡眠临界区。用户 `mutex/condvar` 的共享状态位于用户地址空间，内核只实现 `futex` 等待和 唤醒，不替用户程序保存业务谓词。

锁设计的审核问题包括：持有什么锁时可能调度；退出是否取消 `waiter`；地址空间销毁是否 与 `futex copy` 竞争；管道 `EOF` 是否唤醒全部相关端；系统调用 `copy` 用户数据时是否可能持 `PMM` 锁。

第9章 ATA 驱动与 MyFS 文件系统

9.1 ATA PIO 与 LBA48 驱动

ATA 驱动以 PIO 方式访问 `QEMUIDE` 磁盘，设置命令寄存器、轮询状态、传输扇区数据并处理超时。PIO 实现简单、适合教学，但 CPU 需要参与数据传输，性能和并发性不如 DMA。

LBA28 在 512B 扇区下约受 128GiB 边界限制。项目实现 LBA48 寄存器写入顺序和地址检查，并使用 256GiB 稀疏镜像把 MyFS 放在旧边界以上验证，避免仅在低地址磁盘上“名义支持”。

9.2 MyFS 磁盘格式

MyFS 块大小为 4096 B，格式版本为 4。磁盘区域依次包含超级块、inode 位图、数据块

位图、inode 表和数据区。具体起点由超级块记录，Guest 不再把各区域固定为某几个常数块。

超级块：超级块包含 magic、版本、块大小、总 inode/块数，以及 inode 位图、数据位图、inode 表和首个数据块的位置/长度。挂载时检查 magic、版本、范围和相互关系，拒绝明显损坏或不兼容镜像。

inode 与目录项：inode 固定 72 B，包括 64 位文件大小、类型、链接计数、11 个直接块、一级/二级/三级间接块和保留 flags。目录项固定 64 B，由 32 位 inode 号和最长 60 字节名称组成。宿主 mkfs 和 Guest 共享同样布局，并用编译期大小断言降低结构漂移风险。

位图管理：inode 位图记录 inode 分配，数据位图记录块分配。创建对象时先找到空闲位并更新相应结构；失败回滚已分配资源。删除文件时释放所有数据/索引块和 inode，不能只清目录项造成泄漏。

9.3 直接和多级间接索引

小文件优先使用 11 个直接块。继续增长时依次使用一级、二级和三级索引，每个 4 KiB 索引块容纳 1024 个 32 位块号。最大文件块数为： $11 + 1024 + 1024^2 + 1024^3$

最大理论字节数再乘 4096，但实际还受文件大小字段、磁盘容量和实现边界限制。

9.4 路径、文件与目录操作

路径解析支持绝对和相对路径，以进程 cwd inode 和 cwd 字符串为上下文逐段查询目录。名称和总路径长度分别受 FS_NAME_MAX=60 与 FS_PATH_MAX=256 限制。根目录 inode 固定为 1。

文件与目录操作：系统实现 open/read/write/close/unlink/list/stat/mkdir/chdir/getcwd 对应能力。读写以 inode、文件 offset 和长度为输入，跨块时逐块解析索引。目录创建生成目录类型 inode 和父目录项。

9.5 文件对象、描述符与管道

每个进程有标准描述符 0/1/2 和普通 FD 表。FD 指向共享 file object；file object 保存 inode、offset、引用和锁。fork/dup 后多个 FD 可共享对象和 offset，close 只减少引用，最后一个引用才释放对象。

dup/dup2 与管道：dup 复制到可用 FD，dup2 复制到指定 FD，并正确关闭原目标。管道使用统一的 file object 接口提供读写端和缓冲。Shell 通过 pipe + fork + dup2 + exec 实现 A | B，通过 open + dup2 实现重定向，因此这些能力不是 Shell 内的特殊字符串演示。

9.6 FS 服务与宿主机 mkfs

inode 位图记录 inode 分配，数据位图记录块分配。创建对象时先找到空闲位并更新相应结构；失败回滚已分配资源。删除文件时释放所有数据/索引块和 inode，不能只清目录项造成泄漏。

宿主机 mkfs 工具：tools/mkfs.c 根据磁盘大小和 FS_START_LBA 计算布局，初始化超级块、位图、根目录，写入 用户 ELF 和字体资源。大镜像使用稀疏文件，避免边界测试真实占满宿主磁盘。

9.7 关键算法、缓存与服务协议

kernel/disk.c 使用主盘、EXT PIO 命令。LBA48 task-file 按 ATA-6 要求先写高字节组，再写 低字节组；扇区数 0 表示 256。读命令为 0x24，写命令为 0x34，写后执行 0xEA flush。

```
select master LBA48

wait BSY clear

write count high + LBA[24..47]

write count low  + LBA[0..23]

issue READ/WRITE EXT

for each sector: wait BSY/DRQ → transfer 256 words

write: FLUSH CACHE EXT
```

每次命令最多 256 扇区，入口检查 48 位地址和 lba + count - 1 溢出。状态轮询有固定上限；失败执行 ATA soft reset，checked 接口最多重试 3 次。disk_lock 串行化 task-file，防止两个 线程交叉编程寄存器。

16 项 write-back 块缓存：

MyFS 有 16 个 4 KiB cache entry，每项记录块号、age、valid、dirty 和 8 字节对齐数据。命中 时更新 age；未命中选择无效或最旧项，若 victim dirty 先写回，再读取目标块。FS 服务串行化 调用，因此缓存元数据无需暴露给多个普通调用者并发修改。

当前策略在每个服务请求完成前 cache_sync() 所有脏块，优先保证 QEMU 重启后的持久性而非 最大吞吐。它不是崩溃一致性日志：若在多个相关块写回中间断电，仍可能出现部分更新；这一点 在局限和后续工作中明确说明。

多级索引寻址算法：

逻辑文件块号 i 的解析区间：

$0 \leq i < 11$ direct[i]

11 <= i < 11 + 1024	indirect_1[i-11]
之后 1024 ² 个	indirect_2[a][b]
之后 1024 ³ 个	indirect_3[a][b][c]

create=false 的读取遇到零指针返回稀疏/未分配结果；create=true 的写入按层分配并清零索引块。若深层分配失败，rollback 函数比较 old/new inode，释放本次新增的叶块和索引树，不能留下只有上层指针却无有效数据的结构。truncate/unlink 递归释放一、二、三级索引树。

路径解析和删除语义：

next_component 逐段解析 /，拒绝过长分量；绝对路径从 root inode 开始，相对路径从 cwd 开始。创建路径需要先解析父目录并确认末段不存在。目录删除前用 dir_empty 检查，防止仍含子项时释放 inode。open 引用 g_open_refs，unlink 与打开对象的释放路径协调 inode 生命周期。

规范化 cwd 字符串处理 .、.. 和重复分隔符，inode 查找结果与可显示路径同步更新。路径解析返回的 inode 类型决定能否 chdir、list 或按普通文件读写。

管道环形缓冲与 EOF：

pipe_object 容量为 4096 B，含 head、tail、count、readers、writers 和 spinlock。读端在缓冲为空且 writers>0 时阻塞；writers=0 时返回 EOF。写端在缓冲满且 readers>0 时阻塞；readers=0 时返回错误。read/write 每次在锁内复制可用部分，状态变化后唤醒对端等待者。

file object 区分 REGULAR、PIPE_READ、PIPE_WRITE。最后一个 pipe 端引用释放时更新 readers/ writers 并唤醒对端；两个端点均归零后回收 pipe object。运行时统计 active file/pipe objects，FS 测试对比前后快照发现泄漏。

FS 服务协议：

fs_request 包含 operation、flags、offset、length、内核 buffer、路径、cwd、inode 和 stat 结果。调用线程通过 IPC 向 FS service 发送固定类型请求，服务执行 fs_service_handle 后同步脏缓存并回复。请求对象和 buffer 在完成前由调用者内核上下文持有，用户指针已提前复制。

支持操作包括 OPEN、READ、WRITE、UNLINK、LIST、STAT、MKDIR、CHDIR、GETCWD、RELEASE。服务线程串行化磁盘结构，但 file object 自身 offset/pipe 状态仍有各自锁。

第10章 TTY、控制台与显示系统

10.1 TTY 总体结构

TTY 向用户 read/write 提供终端抽象，连接键盘事件、输出渲染、控制台状态和前台进程。IRQ 路径只做有界输入处理，可能阻塞的用户请求通过队列和线程上下文完成，避免中断处理器睡眠。

10.2 键盘输入

键盘模块读取扫描码，维护 Shift/Ctrl/Alt/Caps 等修饰状态，解析普通键和扩展键，再把字符或控制事件交给 TTY。系统主要支持 ASCII，不宣称支持 Unicode 输入法。

8042 缓冲排空：旧路径一次 IRQ 只读一个数据字节，快速自动输入和 framebuffer 渲染叠加时可能漏掉后续扫描码。当前 ISR 循环检查状态端口，最多处理 64 个就绪字节，再发送 PIC EOI。上限避免异常硬件状态导致中断中无限循环。

输入队列：扫描码解析结果进入内核队列，前台 read 在无数据时阻塞，输入到达后唤醒。队列访问需要短临界区保护；系统调用在等待时不能持有会阻止 IRQ 或生产者前进的锁。

10.3 虚拟控制台与滚屏

三个虚拟显示控制台，分别保存显示和滚屏状态。切换只改变显示目标，不销毁其他控制台内容。自动测试以 VGA 文本快照核对可观察状态。

滚屏历史：终端保存超过当前屏幕的历史，PageUp/PageDown 浏览，新的输出和光标定位按控制台状态更新。历史浏览与当前输入行需协调，避免重绘后丢失 prompt 或编辑内容。

10.4 VGA、framebuffer 与字体渲染

VGA：VGA 后端使用 80×25 文本缓冲，复杂度低，可作为 framebuffer 不可用时的回退。多数自动测试显式选择 VGA，以稳定抓取文本并减少像素渲染对测试速度的影响。

framebuffer 图形终端：默认交互后端由 kernel/qemu_fb.c 扫描 PCI 上的 QEMU/Bochs VGA 设备，读取 BAR 大小，设置 Bochs 显示寄存器并把线性 framebuffer 映射到 0xFFFF900000000000。渲染器按构建配置的宽高和 32 bpp 绘制背景、字形和光标。Makefile 支持 1024×768、1280×720 和 1440×900。

PSF 字体渲染链路：assets/fonts/terminal.psf → tools/mkfs.c 写入 MyFS → kernel/qemu_fb.c 运行时从 MyFS 读取。

10.5 终端编辑与前台进程控制

支持 Ctrl+L 清屏、Ctrl+U 清行、Ctrl+W 删除前一单词、方向键、Home/End、PageUp/PageDown、F1/F2/F3、历史和 Tab 补全。Shell editor 测试通过 monitor 发送按键并检查最终命令结果。

前台进程控制：TTY 记录前台 PID，Ctrl+C 请求终止当前前台任务。普通子进程的内部 fork/wait 不能随意覆盖 Shell 建立的前台控制关系。当前机制足以支持基础前后台执行，但不等同于完整 POSIX 进程组、session 和 job control。

10.6 键盘事件与 TTY 输入服务

键盘层有 4096 项 keyboard_event 环形队列，事件类型包括字符、特殊键、控制台切换和滚屏。扫描码解码维护 Shift、CapsLock、Ctrl 与 E0 扩展状态；方向键/Home/End/Delete 转为 ANSI 风格序列交给 Shell editor。F1 ~ F3 直接变为控制台切换事件。

TTY 层另有字符输入队列。键盘事件队列满时优先保留回车和控制键；TTY 字符队列满时同样可丢弃最旧普通字符以保留命令结束、EOF、清行和前台终止事件。这是过载降级策略，不代表无限输入不丢失；正常压力测试应在队列容量内保持完整顺序。

用户 read(0, ...) 经系统调用进入 TTY input service。服务接收 IPC 请求后，若字符队列为空，关闭本地中断覆盖“检查队列→设置 BLOCKED”，再调度；IRQ 入队后唤醒服务线程。取得第一个字符就满足阻塞读，随后尽量批量取走当前已到达字符，减少每字符一次 IPC 和回显造成的请求洪泛。

input service 的回复缓冲由请求 mutex 串行保护，调用者收到回复后再复制到系统调用内核缓冲，最后 copyout 到用户空间。输出路径因为 syscall 已完成 copyin，可直接调用有限 ANSI SGR 解析与渲染；代码中保留 output service 结构，但当前 tty_service_write 走直接渲染快路径。

10.7 ANSI、历史与重绘模型

用户输出只解析有界的 ANSI SGR 颜色序列，其他 CSI 命令被安全忽略，超长或非法序列进入 discard 状态直到终止字符，避免把控制字节当普通字符污染终端。内核日志仍使用 raw color API，不经过用户 ANSI 解析。

每个 console 保存最多 history_rows × columns 个 16 位 cell、cursor、line_count 和 view_top。输出时默认跟随底部；PageUp 后 view_top 与 live cursor 分离，新输入可恢复 live view。历史满时整行上移并清空末行。

10.8 framebuffer 渲染与前台控制键

qemu_fb_initialize() 扫描 PCI device 0 ~ 31，匹配 vendor/device 1234:1111 和 display

class。它通过 BAR size probe 得到 framebuffer 物理地址/大小，开启 PCI memory decoding，再用 Bochs 端口 0x1CE/0x1CF 设置宽、高、32 bpp、virtual width 和 linear framebuffer。

framebuffer 被映射到 QEMU_FB_VADDR=0xFFFF900000000000，PTE 带 PWT/PCD，避免普通 cache 策略错误处理 MMIO。字体从 MyFS 的 terminal.psf 读取，检查 PSF2 magic、至少 256 glyph、12×24 和每 glyph 48 B，资源总量限制 64 KiB。

几何计算扣除 24 px 边距，再得到 columns、visible rows 和居中 origin。渲染器保留上次可见 cell 和 cursor，仅重画内容变化或光标移入/移出的格子，避免每次编辑都重写整个 uncached framebuffer。qemu_fb_clear 才遍历全屏像素。

前台控制键语义：Ctrl+C、Ctrl+\、Ctrl+Z 分别请求状态 130、131、148。若存在前台 PID，TTY 把请求交给该进程；无前台任务时，控制字符进入 Shell editor。由于内核没有 stopped 状态，Ctrl+Z 明确降级为 终止前台任务，而不是 POSIX suspend。Ctrl+D 可作为输入控制事件，具体 EOF 行为受当前 read/Shell 处理限制。

第11章 用户态、libc 与 Shell

本章目的：说明用户程序入口、系统调用封装、TLSSerrno、mmap 型分配器、Shell 语法/编辑器、管道重定向和 guidedtour 的用户态证据意义。

11.1 用户程序启动与系统调用封装

内核 ELF Loader 创建用户地址空间、映射程序段与栈，并把入口、argc/argv 布置到初始现场。切换到 Ring 3 后，用户启动代码调用 main；返回值通过 exit 系统调用交给内核。

crt0 与参数传递：usr/crt0.S/相应用户启动路径负责从初始栈取得 argc、argv，满足 C 入口调用约定。系统测试 验证外部命令参数，而不是只运行无参数 hello 程序。

系统调用封装：usr/syscall.h 把 36 个系统调用编号包装为 C 内联接口，统一寄存器约定和返回值。用户程序 不包含内核头部私有结构，只共享稳定 ABI 所需常量和用户可见数据结构。

11.2 libc

usr/libc.c，usr/thread_runtime.h，usr/syscall.h 提供字符串、内存、格式化输出和用户线程同步等基本能力。它只覆盖当前用户程序 所需子集，不宣称兼容完整 ISO C/POSIX libc。

11.3 Shell、内建命令与外部程序

Shell 循环读取一行、编辑与解析 token，识别内建命令或建立外部程序执行计划。执行

计划包括 argv、输入输出重定向、管道段和后台标志。前台任务等待结束后恢复 prompt。

内建命令：内建命令包括 help、about、cd、清屏、文件操作快捷命令、run、demo 等。cd 必须在 Shell 进程自身执行，因为在子进程改变 cwd 不会影响父 Shell。

外部 ELF 程序：用户程序包括 hello、fault、IPC/exec/uaccess/thread/sync/VM/FS/showcase/libc 演示，以及 ls、cat、echo、mkdir、rm、pwd、ps、sleep、kill 等工具。外部程序经 MyFS 读取和 ELF Loader 启动。

11.4 管道、重定向与后台任务

对当前支持的单级 A | B，Shell 创建 pipe 并 fork 左侧执行流：左侧关闭读端、把写端 dup2 到 fd 1 后运行 A；原 Shell 关闭写端、把读端 dup2 到 fd 0 后运行 B，随后关闭临时 fd 0 并等待左侧。B 若是外部命令会再经普通外部命令路径 fork/exec，若是内建命令则临时在 Shell 上下文执行。> 打开/创建输出文件并替换 fd 1，输入重定向替换 fd 0。所有不使用端必须关闭，否则读端可能永远收不到 EOF。当前解析器只支持一个管道分隔符，不是任意长度 POSIX pipeline。

后台任务：命令末尾 & 时，Shell 启动任务后不做同步前台 wait，立即恢复 prompt。后台任务仍可通过 ps/kill 观察和终止。当前没有完整 fg/bg/jobs 和停止/继续信号语义。

11.5 Shell 行编辑、历史与补全

Shell 保存最近命令历史，方向键选择，Tab 按已知命令/路径能力补全，Ctrl 组合键由 TTY/Shell 协作处理。编辑器需要维护光标位置和缓冲内容，重绘不能改变实际命令字符串。

11.6 用户态工具

工具程序刻意复用公开系统调用，不通过展示专用内核接口。ps 获取进程快照，文件工具使用 open/read/write/stat 等接口，sleep/kill 验证生命周期。这样 Shell 演示同时覆盖 ABI 和内核模块协作。

11.7 Shell 解析与外部命令执行

当前 parser 不是通用 AST，而是按固定优先顺序扫描：末尾 & → 第一个 | → < → >/>> → 内建命令 → 外部命令。它支持单级管道和单个重定向场景，不支持引号、转义、变量展开、命令替换、多个管道、重定向任意组合或 POSIX 运算符优先级。报告和演示命令必须限制在已支持语法。

后台实现先去掉末尾 &，fork 子进程递归执行剩余命令，父 Shell 不 wait。由于没有完整 job table，后台完成信息和 fg/bg 管理能力有限。

外部命令、argv 与前台 PID：外部命令路径把空格分隔 token 组装为 argv，fork 后子进

程 exec; 父 Shell 设置 TTY 前台 PID 为子进程、wait 完成后恢复控制。run/exec 内建分别展示新进程和替换映像语义。ELF loader 限制参数数量和单页初始栈容量, 拷贝字符串后按 8 B 对齐, 压入 NULL、argv 指针和 argc。

第12章 系统测试与结果分析

12.1 测试目标与框架

测试目标不是证明系统绝对无缺陷, 而是在明确环境内验证关键不变量和用户可见行为: 构建制品 布局正确、系统能进入 Ring 3、用户错误隔离、并发不丢更新、文件跨重启存在、大 LBA 不截断、快速输入不漏字、framebuffer 尺寸和像素符合预期。

测试框架设计: tests/manifest.sh 注册 suite、case、profile、超时和执行函数; tests/run.sh 负责筛选、固定 seed、重复、超时、临时工作区和结果分类。公共库封装 QEMU、monitor、磁盘镜像、截图、断言 和 artifact。每个 case 至少保存:

command.txt	实际执行函数、超时和 seed
environment	宿主和工具环境
test.log	测试输出
result.env	suite/case/seed/exit/status
work/	失败时可保留的临时镜像和日志
*.ppm / vga.txt	需要时保存视觉证据

状态分为 PASS、FAIL、TIMEOUT、ERROR 和 INFRASTRUCTURE_ERROR。基础设施错误或超时不能改写为 PASS, 也不能未经复现就直接认定为 Guest 内核缺陷。

12.2 fast/full/stress 分层

fast = 快速构建与 mkfs 基础门

full = fast + 有界功能/集成回归

stress = full + 长时间压力 case

执行命令:

```
make test-fast
```

```
make test PROFILE=full SEED=20260821 KEEP_FAILED=1
```

```
make test CASE=sync.stress SEED=20260821 KEEP_FAILED=1
```

full 包含 fast, stress 是完整超集。同步核心门为 10 轮, 独立 stress 为 100 轮。

12.3 构建、启动与存储测试

目的: 在运行 QEMU 前检查静态制品和布局。

步骤: 构建 MBR、Loader、内核、用户 ELF、mkfs 和 fs.img; 检查 MBR 512 B/签名、Loader 大小、内核扇区、物理末端、ELF 和 FS。

预期: 所有边界在允许范围内。

结果: build.artifacts PASS。

动态 Loader 测试:

目的: 验证内核超过旧固定窗口和 ATA 255 扇区单命令上限后仍被完整加载。

步骤: 构造 300 扇区测试 kernel, 重新组装 Loader, 写临时磁盘, 启动 QEMU。

预期: 分块读取完成, 进入 Ring 3 Shell。

结果: boot.dynamic-loader PASS。

文件系统测试:

目的: 验证 FS 服务、目录、文件、共享 FD、dup、pipe、资源释放和持久化。

步骤: 在独立镜像执行创建/写/读/stat/list、层级路径、重定向与 pipe, 退出 QEMU 后复用磁盘再次启动。

预期: 内容、类型、大小和目录结构正确, proof 跨重启存在。

结果: fs.service、tty.shell PASS。

LBA48 边界测试:

目的: 证明寻址超过 LBA28 边界。

步骤: 生成 256 GiB 稀疏磁盘, 把 MyFS 放在 128 GiB 以上, 分别由 mkfs 和 Guest 启动访问。

预期: 地址计算不截断, 系统挂载并启动。

结果: mkfs.lba48、lba48.boot PASS。

12.4 用户态、输入与虚拟内存测试

目的: 验证 crt0、libc、argv、外部命令、pipe、重定向、后台及编辑器。

步骤: 通过 QEMU monitor 发送真实按键, 检查用户程序输出、文件结果、prompt 恢复和编辑后的命令字符串。

结果: userland.core、shell.editor、tty.shell PASS。

输入压力测试:

目的: 验证 Ctrl+C、快速按键和命令顺序。

背景：测试曾观察到 scroll-after 变为 scroll-fter，属于真实漏字。

措施：修复 8042 ISR 后，以固定 seed 重复 userland.core 3 次和 input.stress 2 次。

结果：分别 3/3 与 2/2 PASS，最终 full 中 input.stress PASS。

COW 和 VM 测试：

目的：验证 fork 后共享起点、写时分离、mmap/munmap 和页错误隔离。

预期：父子值互不污染；合法的向下增长栈页可访问；非法用户访问只结束进程。

结果：vm.cow PASS，showcase 的 COW 与 fault 步骤 PASS。

12.5 同步、显示与 guided tour 测试

目的：验证 mutex、condvar、futex、TLS、join/detach 和生命周期。

full 配置：10 轮 × 4 线程 × 每线程 20,000 次。

stress 配置：100 轮 × 4 线程 × 每线程 20,000 次，超时 1800 秒。

结果：sync.core PASS，sync.stress PASS。

framebuffer 测试：

目的：确认默认图形终端不是只编译未运行。

步骤：以 1280×720 启动，monitor 导出 PPM，检查 P6 头、宽高、字节长度和背景/非背景像素。

结果：framebuffer.core PASS，生成 framebuffer-1280x720.ppm。

guided tour 测试：

目的：在用户态通过公开 syscall 一次覆盖系统、COW、IPC、线程、FS 和故障隔离。

步骤：运行 demo，检查 6 个 PASS 和汇总；读取 proof；重启后再读；保存 VGA/主题截图。

结果：showcase.core PASS，生成 terminal-theme.ppm。

第13章 总结与展望

13.1 项目完成情况

MiniOrangeOS 已形成从 BIOS 自举到 Ring 3 用户程序的课程级操作系统闭环。系统使用自己的 512 字节 MBR 和二级 Loader，从 ATA 磁盘装入内核，完成 E820 内存探测、A20 开启、临时页表建立、x86-64 长模式切换和高半区内核跳转。构建过程根据 kernel.bin 的实际大小计算加载扇区数，并同时检查内核磁盘区域与包含 BSS 的物理内存上界，从而避免内核增长后静默覆盖 MyFS 或启动页表。

进入内核后，系统建立 GDT、TSS、IDT、PIC、时钟和键盘中断，提供物理页帧、四级页

表、独立 用户地址空间、VMA、匿名 mmap/munmap、用户栈按页增长和写时复制。进程与线程分别承担 资源管理和执行调度职责，并实现 spawn/fork/exec/exit/wait、用户线程、TLS、抢占调度、 进程观察和终止。

Ring 3 程序通过 36 个系统调用访问内核服务。系统实现消息 IPC、futex、用户态 mutex/条件变量、 文件描述符、dup/dup2 和管道；存储层实现 ATA PIO LBA48、MyFS v4、层级目录、直接及三级 间接索引、write-back 块缓存和宿主机 mkfs；交互层实现键盘扫描码处理、TTY、三个虚拟控制台、 滚屏历史、VGA 回退和 QEMU/Bochs framebuffer。用户态提供启动代码、轻量 libc、Shell、文件 工具和 guided tour 演示程序。

13.2 主要成果与技术特点

本项目的主要成果不是若干彼此孤立的功能，而是多个跨模块闭环：

1. 启动闭环：构建系统计算实际内核尺寸，MBR 完成首批读取，Loader 分块加载尾部并进入长模式。
2. 隔离闭环：GDT/TSS、用户页表、系统调用入口、用户指针复制和异常处理共同限制 Ring 3 权限。
3. 进程闭环：ELF 装入、调度、fork、COW、exec、exit/wait 和资源回收能够组合运行。
4. 并发闭环：线程调度、IPC、futex、mutex、条件变量和退出取消共同处理阻塞与唤醒。
5. 存储闭环：ATA LBA48、MyFS、文件对象、描述符和 Shell 文件操作支持跨重启持久化。
6. 交互闭环：键盘 IRQ、事件队列、TTY 输入服务、Shell 编辑器和显示后端构成完整输入输出路径。
7. 验证闭环：固定 seed、临时镜像、QEMU monitor、截图、边界用例和压力用例形成可复盘证据。

相较《Orange'S：一个操作系统的实现》的 32 位教学路线，本项目保留“从引导到可交互系统”的 学习方法，但围绕 x86-64 长模式、独立地址空间、SYSCALL/SYSRET ABI、COW、LBA48、层级文件 系统和自动测试重新组织实现。项目不把参考书中的模块名称或设计思想等同于本项目的原创代码。

第14章 功能—文件—函数定位表

14.1 启动与内核基础设施

功能	文件与函数/标签	说明或验证入口
BIOS 第一阶段入口	boot/mbr.S::start	初始化实模式环境并读取首批扇区
MBR 单批磁盘读取	boot/mbr.S::rd_disk_m_16	从 LBA2 读取 Loader 窗口和内核前 42 扇区
Loader 入口	boot/loader.S::loader_start	接管首阶段加载结果
E820 内存探测	boot/loader.S::e820_loop_start、 boot/loader.S::e820_done、boot/loader.S::e820_failed	收集或处理 BIOS 内存映射
Loader 分块加载	boot/loader.S::rd_disk_many_16、 boot/loader.S::rd_disk_m_16	支持超过 255 扇区的内核尾部读取
保护模式与长模式	boot/loader.S::p_mode_start、 boot/loader.S::long_mode_start	建页表并切换 x86-64 长模式
内核链接布局	kernel/linker.Id::KERNEL_VMA、 kernel/linker.Id::KERNEL_LMA	规定高半区虚拟地址和物理加载地址
BSS 清零与内核总入口	kernel/kernel.c::clear_kernel_bss、 kernel/kernel.c::kernel_main	在使用全局状态前清零 BSS 并按依赖初始化
动态 Loader 测试	tests/suites/boot.sh::suite_boot_dynamic_loader	构造 300 扇区内核验证分块加载
GDT/TSS	kernel/gdt.c::gdt_init、kernel/gdt.c::write_tss、 kernel/gdt.c::set_tss_rsp0	建立内核/用户段和 Ring 3 切栈信息
IDT	kernel/idt.c::idt_init、kernel/idt.c::set_idt_gate	安装异常和 IRQ 入口
页错误	kernel/idt.c::isr14_page_fault	分派 COW、栈增长或非法访问

用户异常隔离	kernel/idt.c::exception_from_user、 kernel/idt.c::terminate_faulting_user	终止故障用户进程而不是使内核panic
时钟与睡眠	kernel/timer.c::timer_init、 kernel/timer.c::timer_interrupt_handler、 kernel/timer.c::thread_sleep_ticks	更新 tick、唤醒和提供抢占点
PIC 初始化	kernel/pic.c::pic_init	重映射 IRQ 并建立 EOI 基础
内核日志	kernel/print.c::print_init、kernel/print.c::print_debug、 kernel/print.c::print_error	启动和故障诊断输出

14.2 内存、进程与线程

功能	文件与函数	说明或验证入口
物理内存初始化	kernel/memory.c::init_phy_mem_map、 kernel/memory.c::reserve_physical_range	根据 E820 建立并保留关键物理区
内核直接映射	kernel/memory.c::build_kernel_direct_map	建立内核访问物理内存所需映射
页分配与释放	kernel/memory.c::alloc_pages_owned、 kernel/memory.c::alloc_page_owned、 kernel/memory.c::free_pages_owned、 kernel/memory.c::free_page_owned	按 owner 管理物理页生命周期
用户映射引用	kernel/memory.c::pmm_acquire_user_mapping、 kernel/memory.c::pmm_release_user_mapping、 kernel/memory.c::pmm_page_refcount	统计 PAGE_OWNER_USER 页的用户 PTE 引用
页表创建与映射	kernel/memory.c::create_page_dir、 kernel/memory.c::map_page、 kernel/memory.c::remap_page	创建用户页表和显式映射
解除用户映射	kernel/memory.c::unmap_user_page、 kernel/memory.c::destroy_user_address_space	解除 PTE 并按引用回收页面
用户范围校验	kernel/memory.c::user_range_is_readable、 kernel/memory.c::user_range_is_writable	校验用户地址、页存在和权限
用户数据复制	kernel/memory.c::copy_from_user、	系统调用

制	kernel/memory.c::copy_to_user、 kernel/memory.c::copy_string_from_user	copyin/copyout 入口
COW 页错误	kernel/memory.c::handle_cow_page_fault	根据引用数复制页面或恢复写权限
VMA 建立	kernel/process.c::process_add_vma	记录合法用户地址区间和权限
匿名映射	kernel/process.c::process_mmap、 kernel/process.c::process_munmap	立即分配匿名页并支持 VMA 拆解除除
用户栈增长	kernel/process.c::process_handle_page_fault	处理 VM_GROWSDOWN 范围内的缺页
进程初始化与查找	kernel/process.c::process_init、 kernel/process.c::process_current、 kernel/process.c::process_find_by_pid	管理进程对象和 PID
进程创建	kernel/process.c::process_create、 kernel/process.c::process_create_loaded	创建内核/已装入用户进程
fork 与地址空间克隆	kernel/process.c::process_fork、 kernel/process.c::clone_user_address_space	复制资源并建立父子 COW 映射
退出与等待	kernel/process.c::process_exit、 kernel/process.c::process_wait、 kernel/process.c::process_reap_orphans	僵尸、父子通知和回收
进程观察与终止	kernel/process.c::process_snapshot、 kernel/process.c::process_request_kill、 kernel/process.c::process_request_terminal_signal	支持 ps、kill 和前台控制键
线程创建	kernel/thread.c::thread_create、 kernel/process.c::process_create_thread	建立内核线程或同进程用户线程
用户线程栈与 TLS	kernel/process.c::allocate_user_thread_stack、 kernel/process.c::process_allocate_thread_tls	为新用户线程分配独立栈和 TLS
调度与抢占	kernel/thread.c::schedule、 kernel/thread.c::thread_yield、 kernel/thread.c::thread_preempt_point	选择 READY 线程并切换执行

汇编上下文切换	kernel/switch.S::switch_to	保存和恢复 callee-saved 现场
阻塞与唤醒	kernel/thread.c::thread_block、 kernel/thread.c::thread_block_with_lock、 kernel/thread.c::thread_unblock	支持等待队列且避免检查—睡眠空窗
join/detach/回收	kernel/thread.c::thread_join、 kernel/thread.c::thread_detach、 kernel/thread.c::thread_reap_detached	管理用户线程生命周期
ELF 装入	kernel/elf.c::elf_load_image_args、 kernel/elf.c::execute_elf_args	映射 ELF 段并构造 argc/argv 栈
进入用户态	kernel/usermode.S::enter_user_mode、 kernel/usermode.S::return_to_user_with_arg、 kernel/usermode.S::resume_user_image	首次进入 Ring 3 及 exec 映像切换
VM/COW 自动测试	tests/suites/vm.sh::suite_vm_cow	验证 fork、COW、mmap 和故障隔离

14.3 系统调用、IPC 与同步

功能	文件与函数	说明或验证入口
用户态 syscall 封装	usr/syscall.h::syscall3	按项目 ABI 放置系统调用号和三个参数
系统调用初始化	kernel/syscall.c::syscall_init	配置 EFER、STAR、LSTAR 和 FMASK
汇编入口与返回	kernel/syscall_entry.S::syscall_entry、 kernel/syscall_entry.S::syscall_return_from_handler	保存用户现场、切换内核栈并返回
系统调用分派	kernel/syscall.c::syscall_handler_impl、 kernel/syscall.c::syscall_handler	分派全部 36 个系统调用
文件描述符安装/查找	kernel/syscall.c::install_file_object、 kernel/syscall.c::get_file_object、 kernel/syscall.c::detach_file_object	支持 open/close/dup/pipe 等调用
argv 复制	kernel/syscall.c::copy_exec_arguments	限制参数数量并复制用户字符串

消息 IPC 初始化	kernel/ipc.c::ipc_init	初始化 IPC 全局保护状态
消息发送与接收	kernel/ipc.c::ipc_send、kernel/ipc.c::ipc_receive	按 PID 或 IPC_ANY 匹配并阻塞/唤醒
IPC 退出取消	kernel/ipc.c::ipc_abort_thread、 kernel/ipc.c::ipc_abort_current	从 sender 队列摘除退出线程并唤醒相关方
futex 初始化	kernel/futex.c::futex_init	建立 64 个哈希桶
futex 等待/唤醒	kernel/futex.c::futex_wait、 kernel/futex.c::futex_wake	二次检查用户值并避免 lost wakeup
futex 退出取消	kernel/futex.c::futex_cancel_thread、 kernel/futex.c::futex_cancel_process	清理线程或地址空间上的 waiter
内核 spinlock/mutex	kernel/sync.c::spinlock_acquire、 kernel/sync.c::spinlock_release、 kernel/sync.c::mutex_acquire、 kernel/sync.c::mutex_release	短临界区和可睡眠临界区同步
用户 mutex	usr/thread_runtime.h::user_mutex_init、 usr/thread_runtime.h::user_mutex_lock、 usr/thread_runtime.h::user_mutex_unlock	原子快路径与 futex 竞争路径
用户条件变量	usr/thread_runtime.h::user_cond_init、 usr/thread_runtime.h::user_cond_wait、 usr/thread_runtime.h::user_cond_signal、 usr/thread_runtime.h::user_cond_broadcast	配合 mutex 循环检查谓词
同步自动测试	tests/suites/sync.sh::suite_sync_core、 tests/suites/sync.sh::suite_sync_stress	分别执行有界回归和长时间压力门

14.4 ATA、MyFS 与文件对象

功能	文件与函数	说明或验证入口
ATA 初始化	kernel/disk.c::disk_init	初始化 PIO LBA48 主盘访问和锁
LBA48 寄存器编	kernel/disk.c::ata_select_lba48、	按高字节组、低字节

程	kernel/disk.c::ata_program_lba48	组写 task-file
单次扇区读写	kernel/disk.c::disk_read_sector_once、 kernel/disk.c::disk_write_sector_once	轮询 BSY/DRQ 并传输 16 位数据
磁盘重试接口	kernel/disk.c::disk_read_sector_checked、 kernel/disk.c::disk_write_sector_checked	失败软复位并最多重试三次
MyFS 挂载	kernel/fs.c::fs_init	校验 magic、版本及布局范围
16 项块缓存	kernel/fs.c::cache_get、kernel/fs.c::cache_flush_entry、 kernel/fs.c::cache_sync	管理 valid/dirty/age 和 write-back
inode 读写	kernel/fs.c::fs_get_inode、kernel/fs.c::fs_put_inode	读写固定 72 字节 inode 结构
位图分配与释放	kernel/fs.c::bitmap_allocate、kernel/fs.c::bitmap_free、 kernel/fs.c::alloc_data_block	管理 inode 和数据块资源
多级索引	kernel/fs.c::inode_data_block	解析直接、一级、二级和三级间接块
索引失败回滚	kernel/fs.c::rollback_new_blocks	释放本次新增叶块和索引块
路径查找与创建	kernel/fs.c::fs_lookup_path、kernel/fs.c::fs_create_path、 kernel/fs.c::canonicalize	支持绝对/相对路径及 cwd 规范化
文件数据读写	kernel/fs.c::fs_read_inode、kernel/fs.c::fs_write_inode	按 offset 跨块读写 inode 数据
删除与回收	kernel/fs.c::fs_unlink_path、kernel/fs.c::truncate_inode、 kernel/fs.c::free_inode_blocks	检查非空目录并递归释放索引树
目录与属性	kernel/fs.c::fs_list_path、kernel/fs.c::fs_stat_path	支持 list 和 stat 系统调用
FS 服务线程	kernel/fs.c::fs_service_init、kernel/fs.c::fs_service_main、 kernel/fs.c::fs_service_handle、kernel/fs.c::fs_service_call	通过内核 IPC 串行化 MyFS 请求
普通文	kernel/file.c::file_regular_create、kernel/file.c::file_retain、	维护共享 offset、引

件对象	kernel/file.c::file_release	用和最终释放
文件对象读写	kernel/file.c::file_read、kernel/file.c::file_write	统一普通文件和 pipe 接口
管道	kernel/file.c::pipe_create、kernel/file.c::pipe_read、kernel/file.c::pipe_write	4096 字节环形缓冲和 EOF/断端语义
宿主机 mkfs	tools/mkfs.c::main、tools/mkfs.c::inode_at、tools/mkfs.c::write_block	计算布局并写入用户 ELF 和字体资源
文件系统自动测试	tests/suites/fs.sh::suite_fs_service	验证目录、文件、FD、pipe、泄漏与持久化
LBA48 自动测试	tests/suites/mkfs.sh::suite_mkfs_lba48、tests/suites/lba48.sh::suite_lba48_boot	在 128GiB 边界以上创建并挂载 MyFS

14.5 键盘、TTY 与显示

功能	文件与函数	说明或验证入口
键盘初始化	kernel/keyboard.c::keyboard_init	初始化 4096 项事件环形队列
扫描码解码	kernel/keyboard.c::keyboard_handle_scancode	维护修饰键、E0 状态并产生字符/特殊事件
键盘事件出队	kernel/keyboard.c::keyboard_pop_event	向 TTY 输入服务提供事件
8042 IRQ 排空	kernel/idt.c::isr33_keyboard	单次 IRQ 有界读取全部就绪扫描码
TTY 初始化	kernel/tty.c::tty_init、kernel/tty.c::tty_use_framebuffer_geometry	创建三控制台和显示几何
字符与滚屏	kernel/tty.c::console_put_char、	更新控制台

	kernel/tty.c::console_scroll_history	cell 和历史
用户输出与 ANSI	kernel/tty.c::tty_write_user、 kernel/tty.c::console_ansi_apply_sgr	有界解析 SGR 并写当前控制台
TTY 输入处理	kernel/tty.c::tty_handle_input_char、 kernel/tty.c::tty_take_input_char	处理控制键、字符入队和读取
TTY 输入服务	kernel/tty.c::tty_service_init、 kernel/tty.c::tty_input_service_main、 kernel/tty.c::tty_service_read	通过服务线程完成阻塞 stdin 读取
控制台切换	kernel/tty.c::tty_switch、kernel/tty.c::flush_active_console	F1/F2/F3 切换并重绘可见控制台
前台 PID	kernel/tty.c::tty_set_foreground_pid、 kernel/tty.c::tty_get_foreground_pid	将 Ctrl+C 等请求发送到前台任务
VGA 回退	kernel/vga.c::vga_apply_terminal_theme、 kernel/vga.c::vga_set_block_cursor	提供稳定文本显示和测试 oracle
QEMU framebuffer 初始化	kernel/qemu_fb.c::qemu_fb_initialize	扫描 PCI、设置 Bochs 寄存器并映射 BAR
PSF 字体加载	kernel/qemu_fb.c::load_font	校验 PSF2 头、字形尺寸和资源上界
framebuffer 增量渲染	kernel/qemu_fb.c::render_cell、 kernel/qemu_fb.c::qemu_fb_render_cells、 kernel/qemu_fb.c::qemu_fb_clear	重画变化 cell、光标或全屏
输入压力测试	tests/suites/input.sh::suite_input_stress	验证快速按键、控制键与命令顺序

framebuffer 测试	tests/suites/framebuffer.sh::suite_framebuffer_core	检查 PPM 尺寸、像素长度和前景/背景
-------------------	---	----------------------

14.6 用户态、Shell 与演示

功能	文件与函数	说明或验证入口
用户程序启动	usr/crt0.S::_start	读取初始 argc/argv 并调用 main
errno/TLS	usr/libc.c::_errno_location、 usr/thread_runtime.h::user_errno_location	为不同线程提供独立 errno 槽
字符串与内存函数	usr/libc.c::memset、usr/libc.c::memcpy、 usr/libc.c::memmove、usr/libc.c::strlen、 usr/libc.c::strcmp	提供用户程序基本运行库
动态内存	usr/libc.c::malloc、usr/libc.c::calloc、 usr/libc.c::realloc、usr/libc.c::free	基于匿名映射管理用户堆块
格式化输出	usr/libc.c::vsprintf、usr/libc.c::printf、usr/libc.c::puts	提供用户态格式化和终端输出
Shell 主循环	usr/shell.c::_start	读取、编辑、解析并执行命令
内建命令	usr/shell.c::run_command、usr/shell.c::command_ls、 usr/shell.c::command_cat、usr/shell.c::command_pwd	在 Shell 进程内分派内建功能
外部程序	usr/shell.c::command_external、 usr/shell.c::command_run、usr/shell.c::command_exec	组合 spawn/fork/exec 和前台等待
单级管道	usr/shell.c::run_pipeline	组合 pipe、fork、dup2 和 exec
输入输出重定向	usr/shell.c::run_redirected、 usr/shell.c::run_input_redirected	替换标准描述符并在结束后恢复
行编辑	usr/shell.c::editor_insert、usr/shell.c::editor_delete、 usr/shell.c::editor_redraw	维护命令缓冲和屏幕光标
历史与搜索	usr/shell.c::editor_store_history、 usr/shell.c::editor_history_previous、	支持方向键和反向搜索

	usr/shell.c::editor_history_next、 usr/shell.c::editor_search_find	
补全与 kill ring	usr/shell.c::editor_complete、usr/shell.c::editor_kill	提供有限命令补全和 编辑快捷键
guided tour 入口	usr/showcase.c::main、usr/showcase.c::report_step	汇总六项用户态端到 端演示
COW 演示	usr/showcase.c::step_cow	验证 mmap、fork、写 时分离和 wait
IPC 演示	usr/showcase.c::step_ipc	验证父子消息往返和 来源 PID
线程演示	usr/showcase.c::step_threads、 usr/showcase.c::showcase_worker	验证线程、TLS、 mutex 和 join 退出码
文件系统 演示	usr/showcase.c::step_filesystem	验证 open/write/read/stat 和 proof 文件
故障隔离 演示	usr/showcase.c::step_fault_isolation、usr/fault.c::_start	验证用户 fault 后系统 与 Shell 继续运行
用户态自 动测试	tests/suites/userland.sh::suite_userland_core、 tests/suites/tty.sh::suite_tty_shell、 tests/suites/shell_editor.sh::suite_shell_editor	验证 libc、argv、Shell 和真实键盘编辑

14.7 测试框架

功能	文件与函数	说明
用例注册	tests/run.sh::register_case、tests/manifest.sh	记录 suite、 case、profile、 timeout 和函数
profile 筛选	tests/run.sh::profile_includes、tests/run.sh::case_selected	fast/full/stress 分层与显式 case 选择
用例执 行	tests/run.sh::run_case	固定 seed、 timeout、隔离 子 Shell 和结果

		分类
QEMU 生命周期	tests/lib/qemu.sh::qemu_start、 tests/lib/qemu.sh::qemu_wait_ready、 tests/lib/qemu.sh::qemu_stop	管理进程、 socket 和退出 清理
monitor 交互	tests/lib/monitor.sh::monitor_send、 tests/lib/monitor.sh::monitor_screendump	发送按键、读 取文本或导出 截图
截图判定	tests/lib/screenshot.sh::check_terminal_theme_screenshot、 tests/lib/screenshot.sh::check_framebuffer_screenshot	验证 PPM 格 式、尺寸和像 素分布
artifact 记录	tests/lib/artifacts.sh::artifacts_init_case、 tests/lib/artifacts.sh::artifacts_record_environment、 tests/lib/artifacts.sh::artifacts_record_result	保存命令、环 境、日志和最 终状态
集成测试	tests/suites/integration.sh::suite_integration_smoke	验证启动、用 户态和服务协 作主链
guided tour 测试	tests/suites/showcase.sh:	

第15章 声明

本人以单人组形式完成 MiniOrangeOS 课程设计，负责项目选题落实、环境搭建、需求分析、架构取舍、源码集成与修改、调试、测试、文档整理和答辩准备。项目参考于渊《Orange'S：一个操作系统的实现》所采用的教学路线，并查阅 x86-64、ATA、QEMU 和工具链相关资料；参考内容主要用于理解操作系统原理、硬件接口和模块组织，不把参考设计思想表述为本人原创。

开发过程中使用了 AI 工具辅助进行方案讨论、代码草拟、错误定位、测试建议、源码审查和文档整理。AI 输出未被视为天然正确，也不等同于本人纯手写代码；进入最终版本的内容由本人结合当前源码进行阅读、修改、编译、运行和测试。